

Data 200s Milestone 2 - Exploratory Data Analysis

Colman Bouton, Alison Jarvis

10/23/2025

1 Introduction

Problem Scope

We are tasked with creating two different models which serve different purposes. The first will classify the type of a disaster, distinguishing between midwest flooding and southern California fires. This will allow the agency to make quick decisions about appropriate subdivisions to deploy to assist with the response effort. The second model serves the purpose of automatically detecting building level damage after a hurricane disaster. This will also help the agency decide how to rapidly allocate resources. To this end, our goals in performing exploratory data analysis are to 1) understand the data provided, along with any potential issues with the data we will have to consider. And 2) determine which features of the images will be useful in developing models which perform the two given tasks.

Data Sampling and Collection

The data for this investigation was provided to us in the format of npz and npy files, which correspond to the disaster images and disaster labels, respectively. These are simple binary formats which are used to store numpy arrays, and as such the data can be loaded into numpy array variables using numpy library functions. Additionally, image metadata, including damage labels, were stored in a companion csv file. The label and image data are arrays of identical lengths, and we assume that the label at a given index corresponds to the image at the same index.

This data has been down-sampled for us, into training and testing sets, and we exclusively investigate the training set for the exploratory data analysis, as we will exclusively use the training dataset to create a model. There may have been some bias involved in the process to separate the dataset into training and testing. We don't know the method used for this division and therefore cannot comment on any biases that may have been present in this process, but we may investigate whether there were biases during model development by resampling the training and testing dataset and using cross-validation. Apart from the training and testing division, we do not further down-sample the dataset for exploratory data analysis. We do, however, crop the images to get rid of the blank portions, as described in later sections.

Data Cleaning

Upon loading the data, we began the process of inspecting what had been loaded to identify any possible irregularities or issues that may need to be addressed. We first looked more into the format and structure of the data. The images consist of color (RGB) satellite images of different disasters. As such, each image is represented by a 1024x1024x3 uint8 numpy array, the first two coordinates of which correspond to a pixel's location within the image, and the third which represents the color of the image in RGB format. The images are labeled according to three categories, SoCal fire, Midwest flooding, and hurricane Matthew. There are additional labels for each disaster type which quantify damage level, on a scale from 0 to 3. Given the above, the granularity of the data is a single satellite image of an area impacted by a disaster, which is unique in location and time.

We also examined the data to identify any potential outliers or corruption within the images. While not invalid per se, each of the datasets contain images with black, non-informative sections. These will need to be accounted for when engineering features and training models. Additionally, the social fire dataset contains a group of 7 outlier images that are entirely or almost entirely over the ocean, rather than the land where destruction from the fire could have occurred. See the section on Gabor Kernels for further information.

The data has little intrinsic structure in the raw files. For preliminary EDA investigations, we load it in as a nested dictionary. The overall dictionary contains subdictionaries for each disaster type. These subdictionaries have two keys, 'images' and 'labels', where the values are arrays of identical lengths containing images and damage labels respectively. For investigations specific to the two sub-problems, we reformatted the data in various ways to make it easier to work with. For the hurricane images, it was useful to create two separate arrays of images corresponding to damage levels 1 and 3. Similarly, when comparing the fire and flooding images, a schema of two arrays, one for each dataset, was helpful.

2 EDA - Basic Statistics

2.1 Image Sizes and Counts

First, in terms of basic statistics, we investigate the number of images for each disaster type. We find that there are 200 images per dataset. The sizes of all of the images, independent of disaster types, were found to be identical. To wit, they are all square images of size 1024×1024 , meaning that they have 1048576 pixels total.

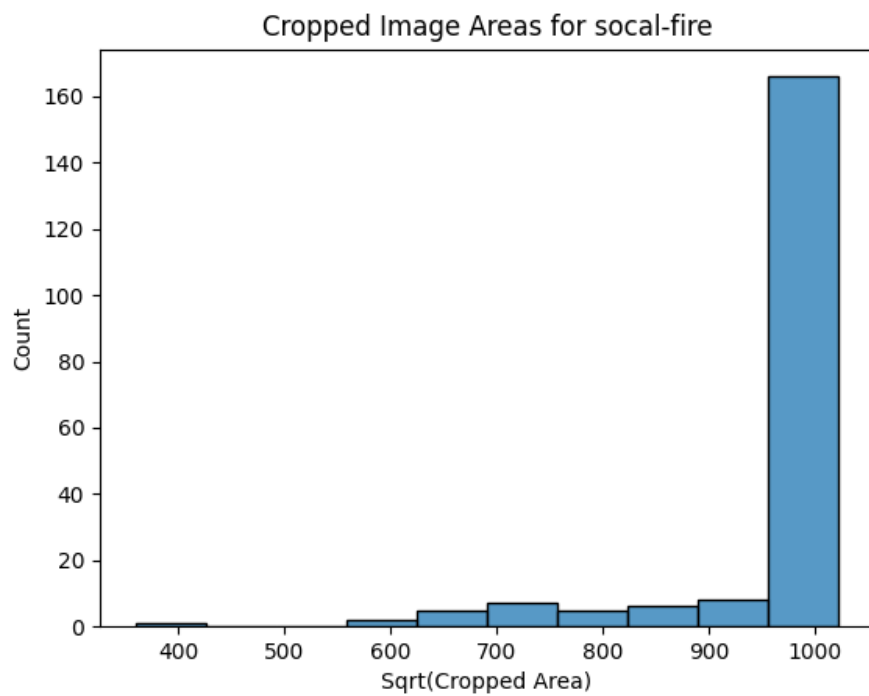
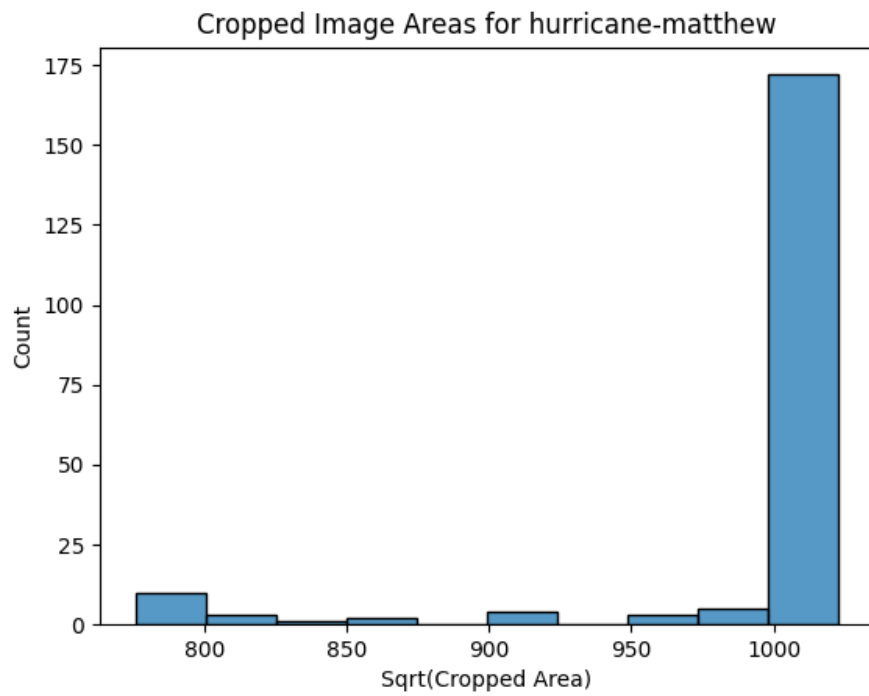
However, upon closer inspection, it would seem that the uniformity in size and shape is due a post-image normalization process. Many of the images in each of our datasets contain a 'blank' portion of the image that is encoded as black pixels (i.e. an rgb value of (0,0,0)). A good example of this can be found in the 24th social-fire image, shown below.

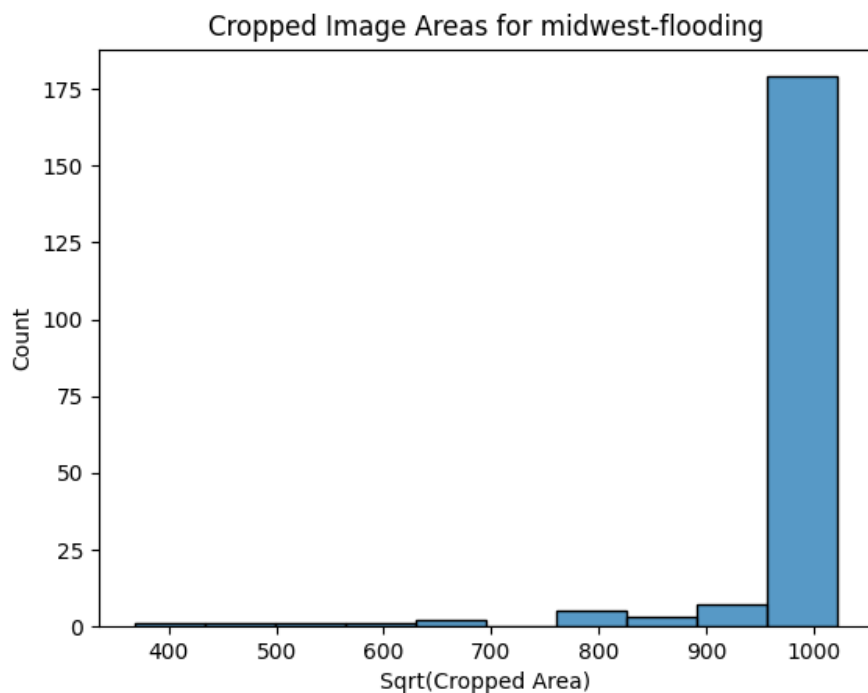
Example Partially Blank Image (social-fire-24)



To attempt to address this issue, we created a utility function to assist with cropping the images. By working with the cropped images, we can avoid some confounding effects introduced by the areas of black pixels. The cropping function also allows us to assess the true size of each image after cropping.

We can use the post-cropping size to examine the degree to which this artifact has impacted our three datasets. Below are three histograms showing the distribution of the square root of true image sizes for each dataset.

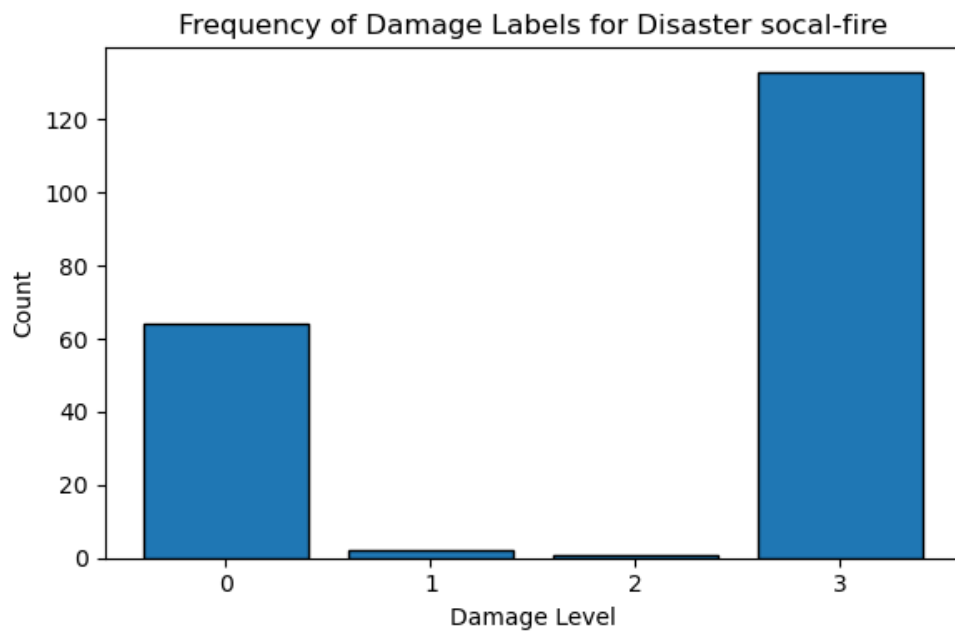
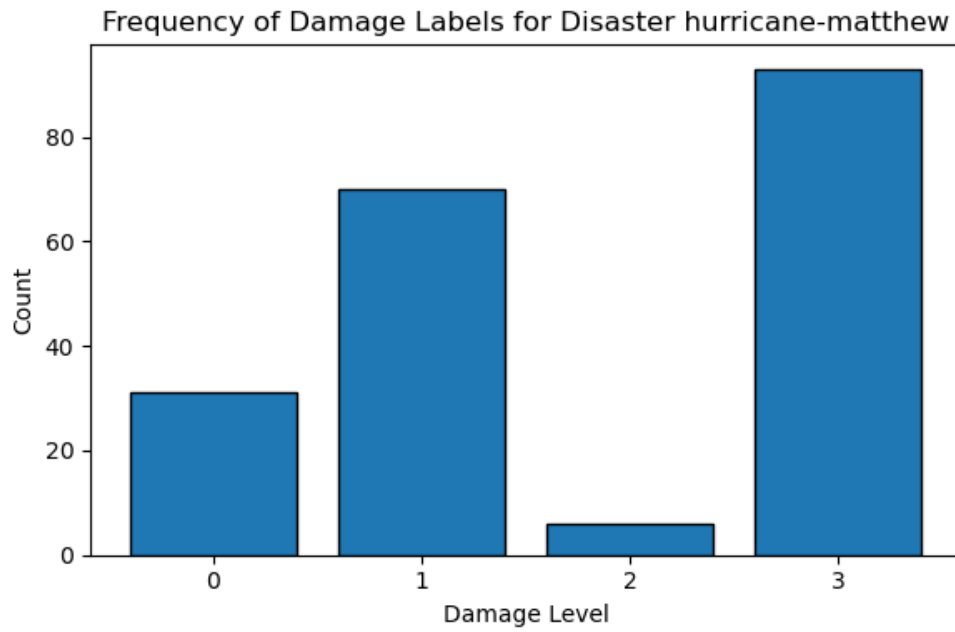


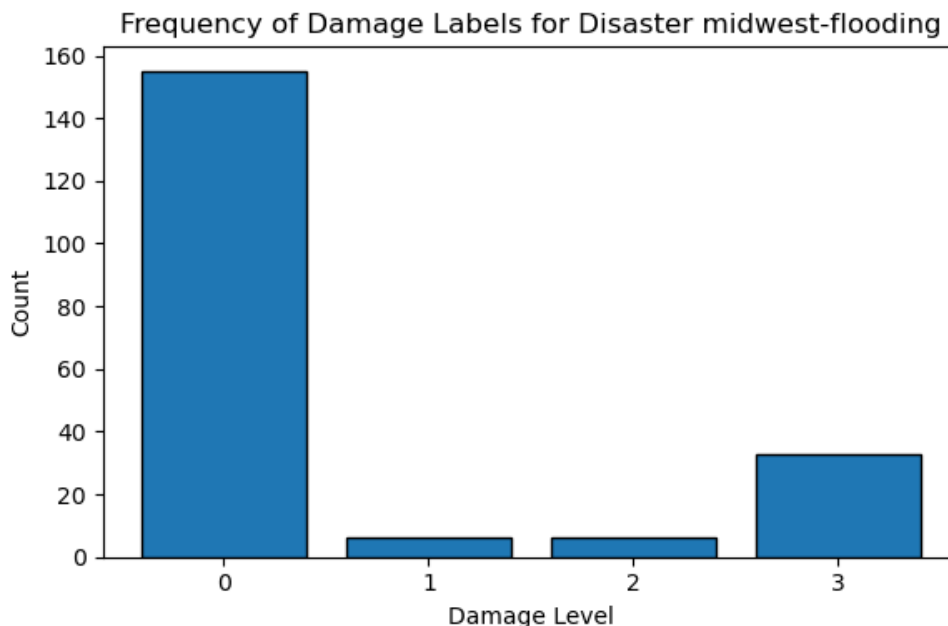


We can see from the above that all datasets have a similar degree of impact by image. It would seem that perhaps the images associated with the hurricane are not as impacted as the other two datasets, as they have a higher average area of valid image overall. We can see from the histograms that this is due to them lacking as intense a left skew as the other two sets of image sizes.

2.2 Label Distributions

The following figures show the distribution of damage labels, visualized as a bar graph of the counts of each damage label for each disaster type.





Social fire and midwest flooding have damage labels that are predominantly either 0 or 3 (the lowest or highest options). Hurricane images, however, have predominantly damage labels of 1 or 3, with a few labels of 0.

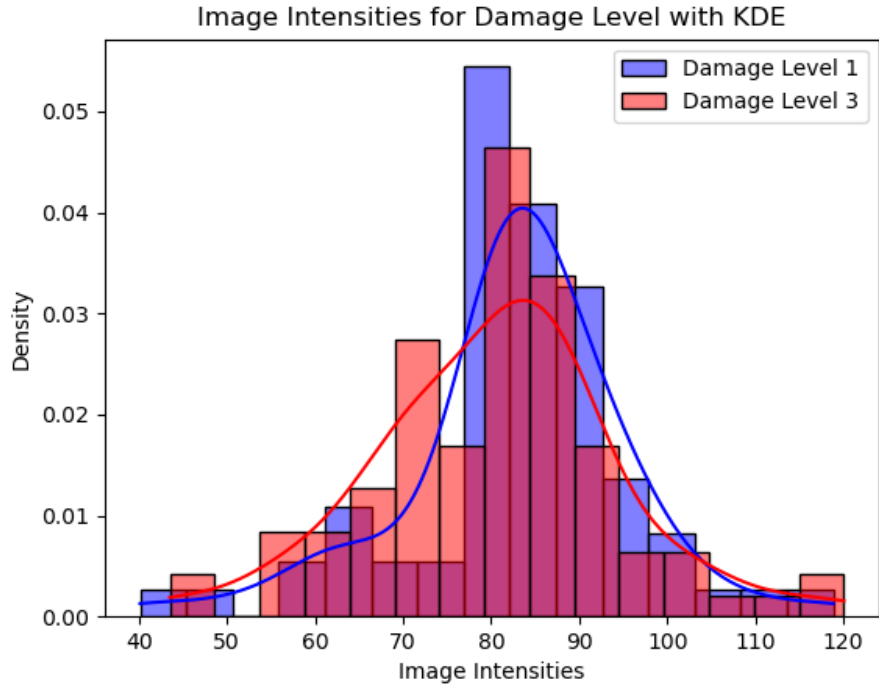
3 EDA - Hurricane Damage Level

Here, we specifically investigate the Hurricane Matthew images, with the ultimate goal of being able to separate hurricane images into classes of damage level 1 and damage level 3. To this end, we attempt to determine whether there are features / parameters specific to damage level 1 or 3 images which can be used to separate the classes. Note that the code corresponding to this section is contained in the notebook *cv_eda_hurricane_damage.ipynb*.

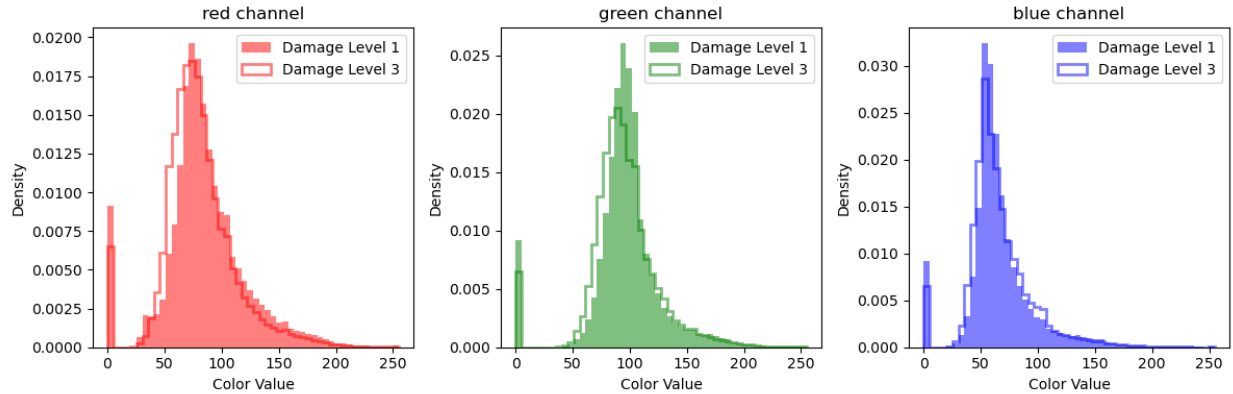
Also note that we do not crop out the corrupted portions of the images in this analysis, given that this issue was less prevalent for the hurricane dataset, and that the percentages of images affected are approximately equal between the two classes. Therefore, if the corrupted portions affect any of the engineered features, they should at least have relatively similar effects for each of the two classes; in other words, we do not expect the corrupted portions to bias separability factors. We may, however, remove these regions during model training.

Color Distribution

We want to try and distinguish whether there are differences in the distribution of color for hurricane images of the damage level 1 class vs damage level 3 class. We first visualize the intensity of the images to determine if this allows us to separate the classes, as shown in the figure 3 below.

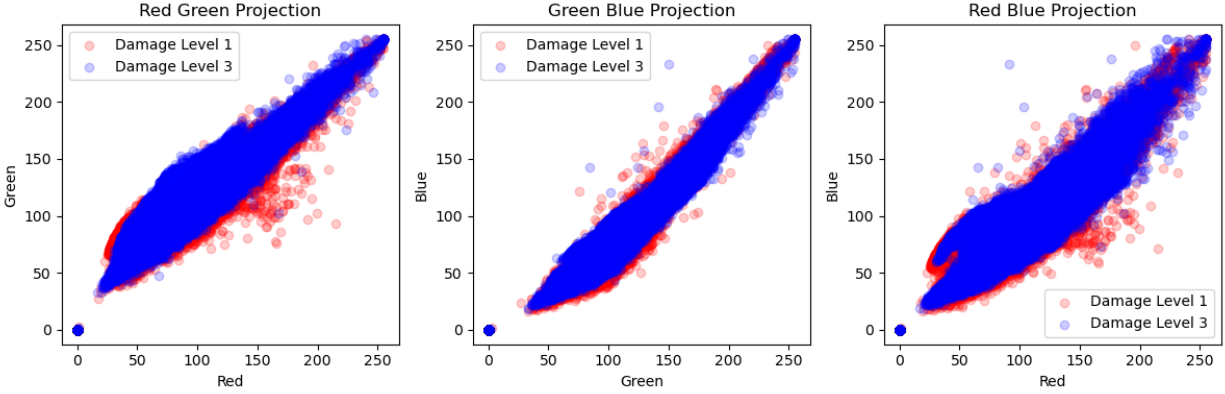


Since the histogram distributions for each population overlap significantly, there is very little difference between the intensities of the images for each damage level, which is mostly expected. We then investigate the distribution of the images in terms of individual colors, red green and blue, for each damage level. We compute this over all pixels of all images for each population.



Unfortunately, there is also very little difference between the color distributions in red, green and blue channels between the two damage levels; there is maybe slightly more red coloring for damage level 1 vs 3, but it is not a significant enough difference as to allow for separation of the classes.

We also want to visualize if there are combinations of colors are specific to one damage level vs another. To assess this, we use a scatter plot of the different permutations of red, green, and blue. As in the previous figure, this is performed for all pixels of all images of each class, but down-sampled to one in every 500 pixels to avoid overcrowding.



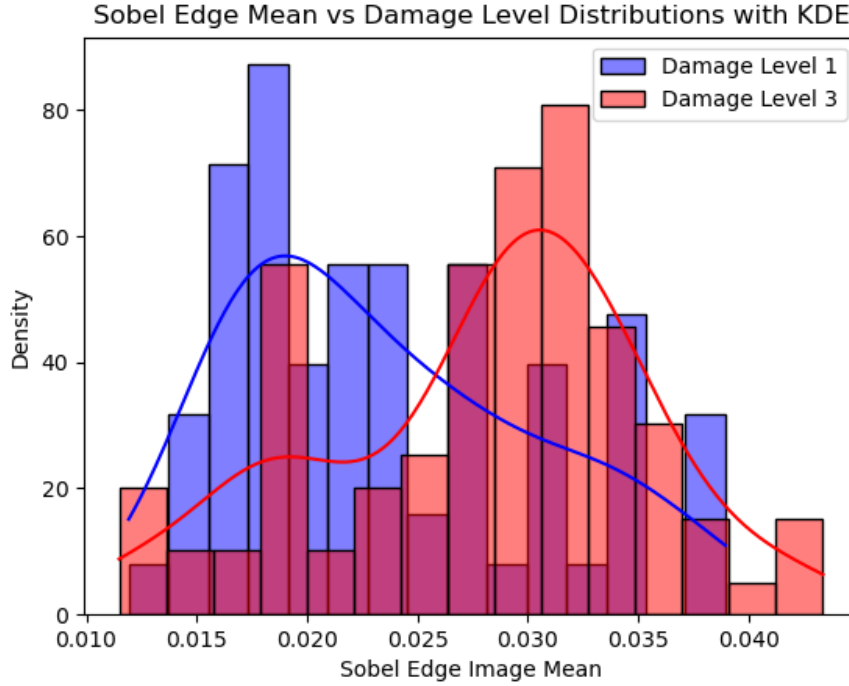
Unfortunately, the populations overlap significantly for each color combination. There is a small group of pixels for damage level 1 that have a higher percentage of red than is present in the damage level 3 population, but this population is not large enough to be sure that it is not a result of one outlier image, and as such is likely not a useful separating feature. We also note the population of pixels at zero in both figure 3 and 3; these are the black, "corrupted" pixels present in some images, which were previously identified. We simply don't consider these pixels in our color analysis.

Other Separating Features

Since we observed that there is nominally no difference in image heights, widths, or overall sizes, shape is not a good separating factor for the damage levels. As we also observed, the classes are not very separable by their color distributions. On the whole this is expected, given that in visualizing the images, the overall color scheme looks relatively similar for damage level 1 and 3 images.

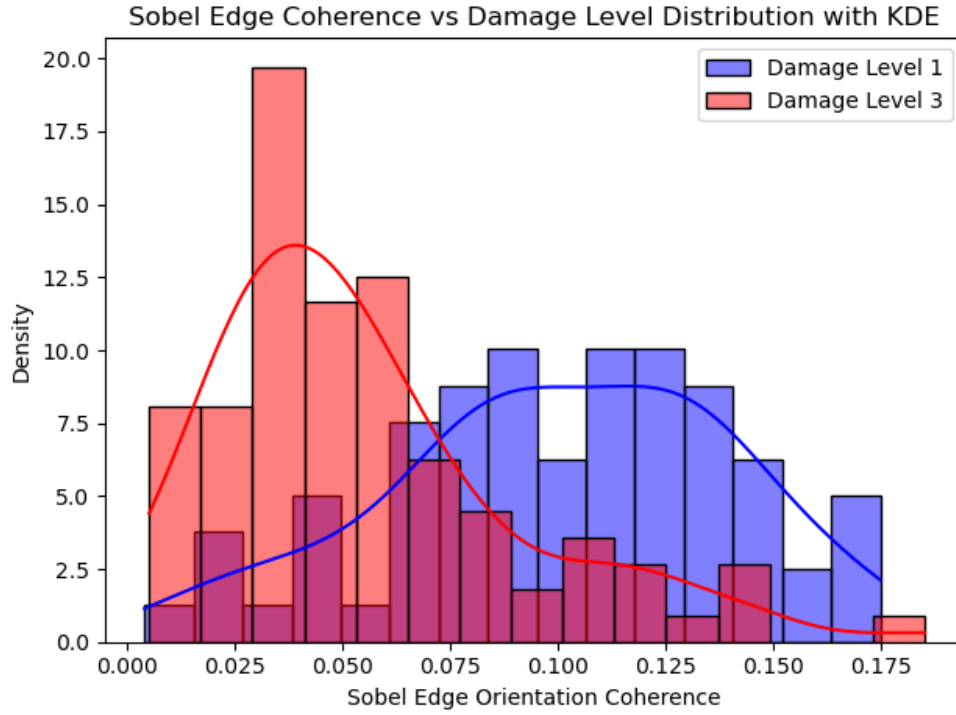
Something we noted from the visual observations of the images of each class, was that in damage level 1 most "lines" were intact, i.e. roads, edges of farmland, etc., whereas for damage level 3, these lines were often interrupted or scattered. This generally makes sense, as infrastructure is ordered and when it is subject to damage it will become disordered. As such, for the next sections we investigate the texture of the images.

Firstly in investigating texture, we use sobel feature detection to get the edges for each image; these essentially represent new images which contain higher values for pixels which constitute "edges". We then get the mean of each sobel edge image, and assess how this distribution varies between the damage level 1 and 3 populations, as seen in figure 3 below.



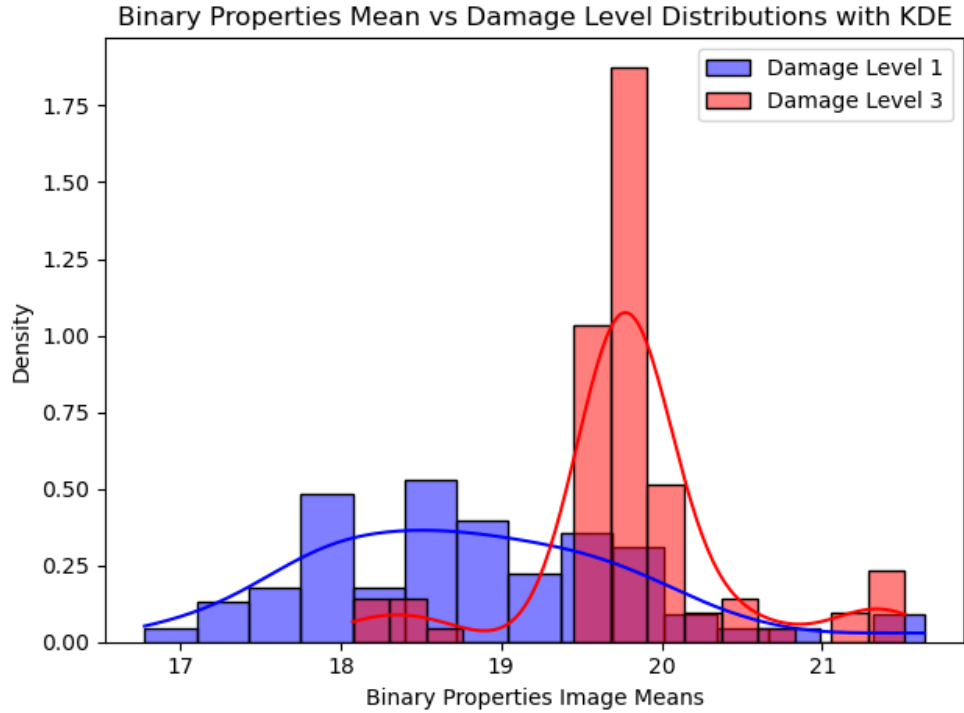
This appears to be a decent separating feature, with the damage level 3 population having on average higher sobel edge image means.

Since we also noticed that the "edges" for damage level 1 have more structure, something we might investigate is the linearity of the edges. To quantify this, we get the sobel edges of the grayscale images, and take the gradient of the edges in the x and y direction, which is used to compute the direction of the gradient. This allows us to then compute the directional coherence, which is a measure of both how linear the edges are and how much the directionality is aligned within a given image (i.e. how parallel they are). We compute the directional coherence for all images for both damage levels, and compare the distributions as histograms in figure 3 below.



This also appears to be a factor which can help separate the damage level 1 and 3 populations; this makes sense given that it is an approximation of how structured the images are.

We next investigate how local binary properties can help distinguish the classes. Local binary properties are essentially a per-pixel measure of texture within a certain defined radius. We calculate the local binary pattern for each image of each class, and take the mean. The histogram below shows the distribution of local binary means for damage levels 1 and 3.

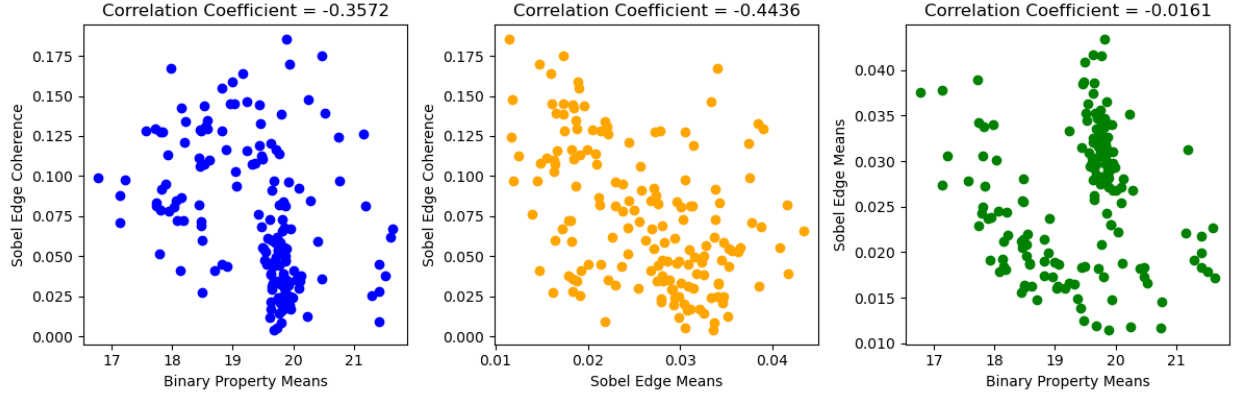


This appears to be a distinguishing feature between the classes as well, which makes sense given that it is a measure of texture. We would expect that texture would tend to be higher in images with higher damage, which is what we observe.

It is worth noting that additional features were explored during the EDA, such as sobel edge lengths, gabor kernel responses, etc., but they were found to not sufficiently separate the populations, and as such are not addressed here. In summary, the three features we found which perform reasonably in separating the damage level 1 and damage level 3 hurricane images include:

1. Sobel Edge Means
2. Sobel Edge Coherence
3. Binary Property Means

A model that we create to attempt to identify the damage level of an image could leverage any or all of these features. However, we also want to investigate whether any of these features are correlated with each other, and therefore redundant. We test this out by creating a scatterplot of the correlation of each of the features, with the correlation coefficient included in the title.



We observe very little correlation in any of the features, meaning that these represent three independent features which collectively can be used to separate the damage level populations. Therefore, we plan to use these features to create a model which will classify the damage level of an image.

4 EDA - Fire vs. Flooding Images

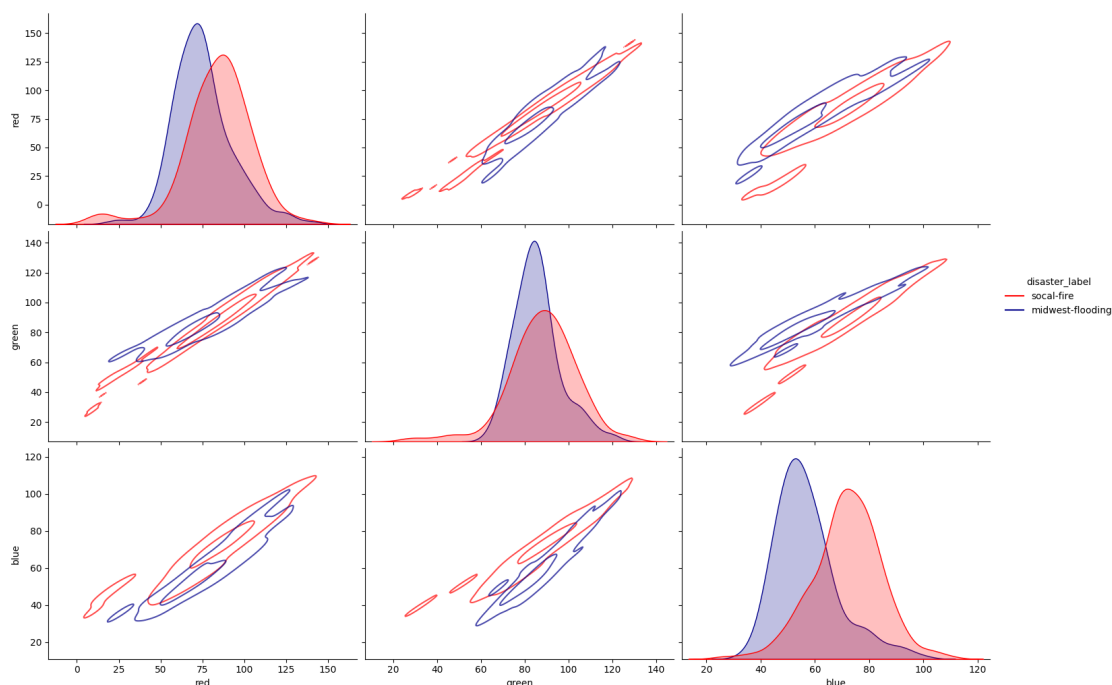
To explore separating flood and fire images, we explored using both color and texture derived features. Namely, we looked at both mean rgb values per image, as well as the mean intensity and variance of each image when transformed via a family of Gabor filters.

Color Distribution

When approaching the use of color to separate flood and fire images, we used the mean rgb values of each image. Intuitively, the colors of an area experience flooding should be broadly different than that of a fire. If true, this can be encapsulated most succinctly by taking the mean color of each image. An added benefit of using the mean as our feature is that it is very easy to account for the blank pixels discovered earlier; we simply normalize the mean by the fraction that are blank.

After calculating our mean colors, we can examine their ability to separate the two classes via a pair plot.

Pair Plot of Mean Red, Green, and Blue Image Values by Disaster Type



From the above, it's clear that the color blue does the best job of separating the two classes. The other two colors do have some useful information, but there is substantially more overlap in the distributions. It's also clear that there is a pretty tight correlation between the color components. However, both green and red do add some additional separability when combined with blue. Thus considering all three means jointly may be fruitful.

Texture (Gabor Kernels)

Given the skimage tutorial, it was clear gabor kernels are most effective when multiple kernel parameters are applied to generate the features. In this eda, we used a family of 16 different kernel parameter combinations. After apply each transformation to an image, we then took the mean and variance of the output's pixel values. This left us with a total of 32 numerical features per image.

This was too many to easily visualize, even with a pair plot, so we attempt to visualize the result by reducing the dimensionality to 2 dimensions using PCA. We took two approaches here. The first used the raw features, without scaling (this seems to be how the skimage tutorial compares theirs), while the second normalized the features used sklearn's RobustScaler class. RobustScaler was selected in particular because there are likely outliers in this dataset.

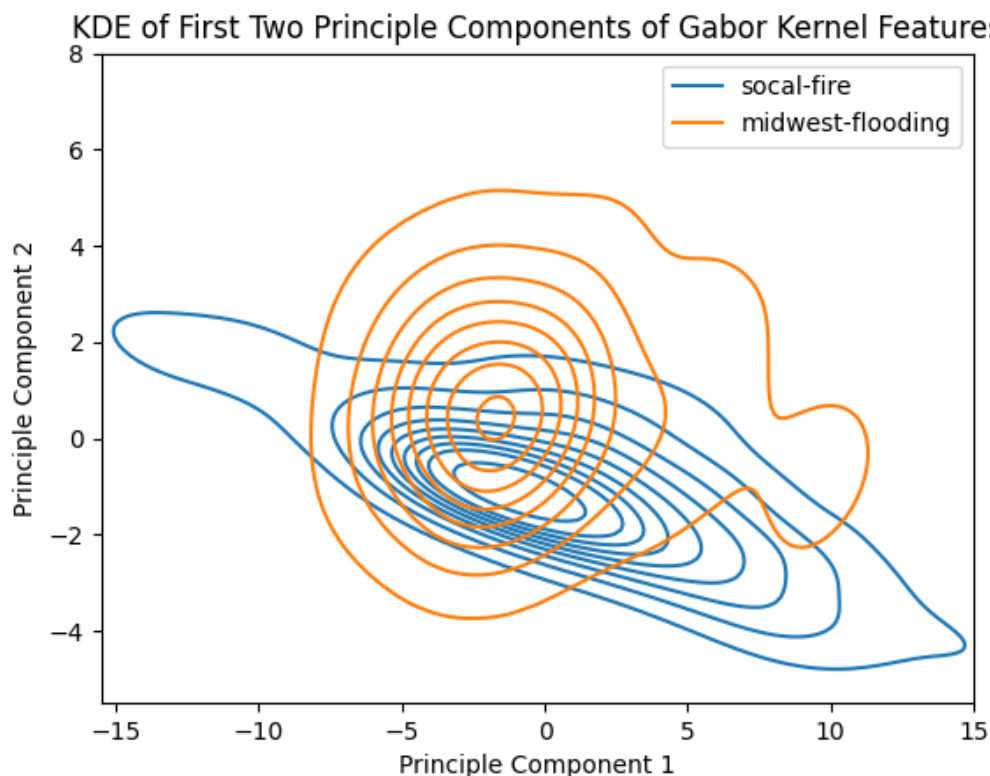
After applying PCA, we check the explained covariance ratios to confirm that the two transformed sets of data still contained the information from the original. We found that 100% of the covariance was explained by the first two PCs for the unscaled dataset, and 69% of the covariance of the unscaled data was explained by their first two PCs. This confirms that a visualization of these will reasonably convey the original data's separation.

We also fitted two logistic regression models to the two components to get a ballpark for how good the separations were. We found an accuracy of 63% for the model trained on the unscaled data, and 83% for the model trained on the scaled data.

This is quite an encouraging result. While by no means the same as testing it against a validation set of data, the fact that such a simple model can achieve this degree of separation is very promising. It also

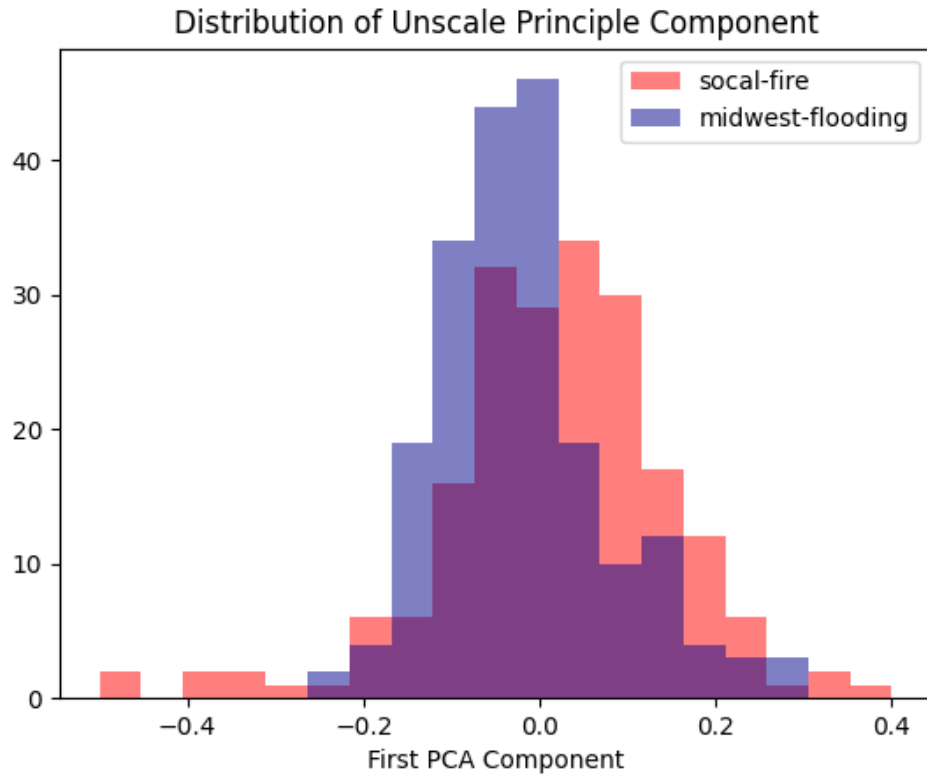
confirms that scaling is important here. Some of the gabor kernels simply have higher average intensities than others, without conveying more information.

Our next step was to visualize the distributions of the two classes over the principle components. The result can be seen below.

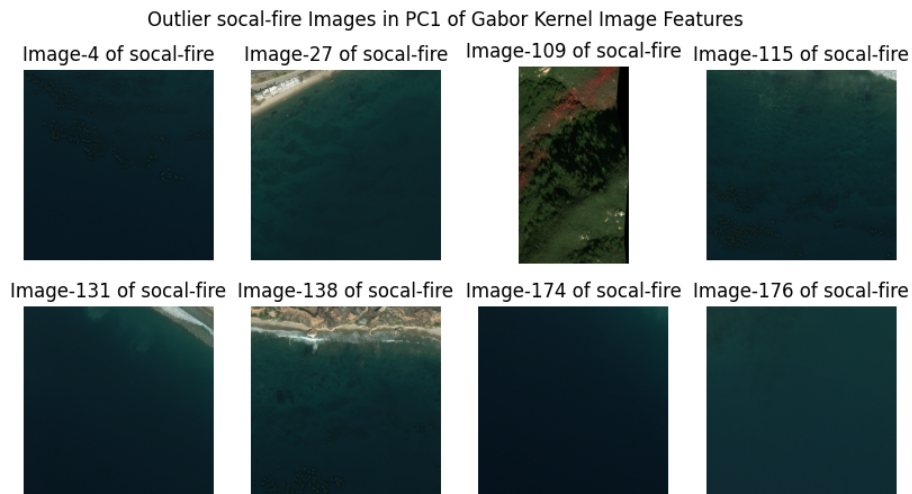


As suggested by the accuracy of the linear regression model, the distributions of the two classes over the first two principle components are decently separated here. We can also see what might be a group of outliers in the fire dataset around the region from -15 to -10 of the first PC.

We also plotted the first principle component of the unscaled data, as although it may be less powerful for predicting the image class, it is easier to visualize over a single dimension (the covariance ratio of the second component is less than 1%).



Interestingly, there appeared to be a small, secondary peak for very low PC1 values of socal-fire. This, in combination with the potentially outliers seen in the previous KDE visualization, prompted us to examine the images in this peak. They are displayed below.



With the exception of Image-109, these are all images consisting mostly of the ocean. We may wish to exclude them from our dataset in the future, as they are not relevant to assessing fire damage.

In conclusion, it appears the features from gabor kernels can prove pretty powerful for separating the images of fires vs. flooding. It also proved useful for identify outliers that we otherwise may not have noticed.